Python & NumPy

Listen verstehen, Zahlenfelder erzeugen, Werte auswählen und berechnen.

01 Python eingebaut / ohne Import

`len(x), type(x)`	Länge und Objekttyp prüfen
`int("3"), float("3.5"), str(3)`	Einzelwerte umwandeln
`x[0], x[-1], x[1:4]`	Erstes, letztes, Element 1 bis 3
`x.append(5)`	Wert an eine Liste anhängen
`sorted(x), sum(x), min(x), max(x)`	Sortierte Kopie / Kennzahlen
`range(5), enumerate(x)`	0 bis 4 / Index-Wert-Paare
`list(zip(x, y))`	Werte paarweise zusammenführen
`[v * 2 for v in x]`	Liste elementweise bearbeiten
`d.get("name", "?")`	Dictionary: Wert oder Ersatz

02 NumPy / numerische Arrays

```
import numpy as np
```

`a = np.array([1, 2, 3], dtype=float)`	Liste in Zahlenarray umwandeln
`a.shape, a.ndim, a.dtype`	Form, Dimensionen, Datentyp
`np.arange(0, 10, 2)`	0, 2, 4, 6, 8; Ende exklusiv
`np.linspace(0, 1, 5)`	5 gleichmäßig verteilte Werte
`np.zeros((2, 3)), np.ones((2, 3))`	Matrix mit Nullen / Einsen
`a.reshape(-1, 1), a.ravel()`	Spaltenmatrix / flaches Array
`a[a > 1], a * 2, a ** 2`	Filtern / elementweise rechnen
`a.mean(), a.std(), a.sum()`	Mittelwert, Streuung, Summe
`np.isnan(a), np.nanmean(a)`	NaN finden / Mittel ohne NaN
`np.sort(a), np.unique(a)`	Sortierte Kopie / eindeutige Werte
`np.concatenate([a, a]), a.copy()`	Arrays verbinden / kopieren

Bei einer 2D-Matrix: mean(axis=0) pro Spalte, mean(axis=1) pro Zeile.
Listen: [1, 2] * 2 wiederholt. NumPy: np.array([1, 2]) * 2 ergibt [2, 4].

pandas

Tabellen laden, prüfen, auswählen und aufbereiten.

03 pandas / Series und DataFrame

```
import pandas as pd
```

`df = pd.read_csv("daten.csv")`	CSV laden; ggf. sep=";" angeben
`df.to_csv("sauber.csv", index=False)`	CSV ohne Indexspalte speichern
`df.head(), df.sample(5, random_state=42)`	Anfang / 5 zufällige Zeilen
`df.shape, df.columns, df.dtypes`	Größe, Spaltennamen, Typen
`df.info(), df.describe()`	Struktur / numerische Statistik
`s = df["alter"]`	Eine Spalte als Series
`df[["alter", "punkte"]]`	Spalten als DataFrame
`df.loc[0:2, ["alter"]]`	Labels; Endlabel inklusive
`df.iloc[0:3, 0:2]`	Positionen; Endposition exklusiv
`df.loc[df["alter"] >= 18]`	Zeilen nach Bedingung wählen
`df.loc[df["alter"] < 18, "gruppe"] = "U18"`	Gezielt Werte setzen
`df["punkte"] = df["punkte"] / 100`	Spalte berechnen / überschreiben
`df.isna().sum()`	Fehlende Werte je Spalte zählen
`df = df.dropna(subset=["ziel"])`	Zeilen ohne Zielwert entfernen
`df["alter"] = s.fillna(s.median())`	Fehlendes Alter durch Median füllen
`pd.to_numeric(s, errors="coerce")`	Zahlen lesen; ungültig wird NaN
`df["name"].str.strip().str.lower()`	Text trimmen und kleinschreiben
`df = df.drop_duplicates()`	Doppelte Zeilen entfernen
`df = df.sort_values("punkte")`	Zeilen aufsteigend sortieren
`df["gruppe"].value_counts()`	Häufigkeit jeder Kategorie
`df.groupby("gruppe")["punkte"].mean()`	Mittelwert je Gruppe
`pd.concat([df1, df2], ignore_index=True)`	Tabellen untereinander hängen
`df1.merge(df2, on="id", how="left")`	Tabellen über Schlüssel verbinden

Bedingungen kombinieren: (df["alter"] >= 18) & (df["punkte"] > 50).

Änderungen meist zuweisen: df = df.drop(...). Für eine eigene Kopie: df.copy().

Visualisieren & ML vorbereiten

Wenige Befehle für Diagramme und den typischen scikit-learn-Ablauf.

04 Matplotlib / Diagramme

```
import matplotlib.pyplot as plt
```

`fig, ax = plt.subplots(figsize=(6, 4))`	Abbildung und Zeichenfläche
`ax.plot(x, y), ax.scatter(x, y)`	Linie / Punktwolke
`ax.hist(x, bins=20), ax.bar(labels, x)`	Verteilung / Kategorien
`ax.set(xlabel="Zeit", ylabel="Wert")`	Achsen beschriften
`ax.set_title("Messwerte")`	Titel setzen
`fig.tight_layout(); plt.show()`	Abstände richten und anzeigen

05 scikit-learn / vorbereiten, lernen, vorhersagen

`X = df[["alter", "punkte"]]`	Merkmale: immer 2D
`y = df["ziel"]`	Ziel: normalerweise 1D
`SimpleImputer(strategy="median")`	Fehlende Zahlen ersetzen
`StandardScaler()`	Merkmale standardisieren
`OneHotEncoder(handle_unknown="ignore")`	Kategorien in Indikatoren wandeln
`prep.fit_transform(X_train)`	An Training anpassen + umwandeln
`prep.transform(X_test)`	Gelerntes auf Testdaten anwenden
`model.fit(X_train, y_train)`	Modell trainieren
`model.predict(X_test)`	Zielwerte / Klassen vorhersagen

Mini-Rezept: numerische Merkmale, Klassifikation

```
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
model = make_pipeline(SimpleImputer(strategy="median"),
                      StandardScaler(), LogisticRegression())
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Erst splitten. Imputer und Scaler nur am Training lernen; die Pipeline hilft dabei.

stratify=y für Klassen. Zeitreihen chronologisch teilen, ohne zufälliges Mischen.

Gleiche Aufgabe, andere Library

Die wichtigsten Übersetzungen und ein kleiner Ausblick auf PyTorch.

06 NumPy / pandas direkt vergleichen

AUFGABE	NUMPY: ARRAY a	PANDAS: SERIES s
Anzahl	`len(a)`	`len(s)`
Erste 3 Werte	`a[:3]`	`s.iloc[:3]`
Werte > 0	`a[a > 0]`	`s[s > 0]`
Mittelwert	`a.mean()`	`s.mean()`
Ohne fehlende Werte	`a[~np.isnan(a)]`	`s.dropna()`
Sortieren	`np.sort(a)`	`s.sort_values()`
Eindeutige Werte	`np.unique(a)`	`s.unique()`
Typ ändern	`a.astype(float)`	`s.astype(float)`

Unterschiede: pandas mean() überspringt NaN; NumPy mean() tut das nicht.
std(): NumPy nutzt ddof=0, pandas ddof=1. Für Gleichheit explizit angeben.
np.unique sortiert; s.unique erhält die Auftretensreihenfolge.

07 Wechseln & schnell zeichnen

`a = df.to_numpy()`	Tabelle → Array, ohne Spaltennamen
`df = pd.DataFrame(a, columns=namen)`	2D-Array → Tabelle mit Spaltennamen
`df.plot.scatter(x="alter", y="punkte")`	pandas-Kurzform für Punktwolken
`df["punkte"].plot.hist(bins=20)`	pandas-Kurzform für Histogramme

08 PyTorch / Tensor-Basics für die späteren Sets

```
import torch
```

`t = torch.tensor(a, dtype=torch.float32)`	Numerische Daten in Tensor kopieren
`t.shape, t.dtype, t.device`	Form, Datentyp und Rechenort
`t.reshape(-1, 1), t.mean(dim=0)`	Form ändern / Mittel pro Spalte
`t = t.to("cpu")`	Tensor auf die CPU übertragen
`t.detach().cpu().numpy()`	Tensor ohne Gradienten → NumPy

Merken: NumPy/pandas verwenden axis, PyTorch verwendet dim.
a.shape ist eine Eigenschaft; a.mean() ist ein Methodenaufruf.